



Chapter 8. *Applying Thread Pools*

[Chapter 6](#) introduced the task execution framework, which simplifies management of task and thread lifecycles and provides a simple and flexible means for decoupling task submission from execution policy.

[Chapter 7](#) covered some of the messy details of service lifecycle that arise from using the task execution framework in real applications. This chapter looks at advanced options for configuring and tuning thread pools, describes hazards to watch for when using the task execution framework, and offers some more advanced examples of using `Executor`.

8.1. Implicit Couplings Between Tasks and Execution Policies

We claimed earlier that the `Executor` framework decouples task submission from task execution. Like many attempts at decoupling complex processes, this was a bit of an overstatement. While the `Executor` framework offers substantial flexibility in specifying and modifying execution policies, not all tasks are compatible with all execution policies. Types of tasks that require specific execution policies include:

Dependent tasks. The most well behaved tasks are *independent*: those that do not depend on the timing, results, or side effects of other tasks. When executing independent tasks in a thread pool, you can freely vary the pool size and configuration without affecting anything but performance. On the other hand, when you submit tasks that depend on other tasks to a thread pool, you implicitly create constraints on the execution policy that must be carefully managed to avoid liveness problems (see [Section 8.1.1](#)).

Tasks that exploit thread confinement. Single-threaded executors make stronger promises about concurrency than do arbitrary thread pools. They guarantee that tasks are not executed concurrently, which allows you to relax the thread safety of task code. Objects can be confined to the

task thread, thus enabling tasks designed to run in that thread to access those objects without synchronization, even if those resources are not thread-safe. This forms an implicit coupling between the task and the execution policy—the tasks require their executor to be single-threaded.^[1] In this case, if you changed the `Executor` from a single-threaded one to a thread pool, thread safety could be lost.

[1] The requirement is not quite this strong; it would be enough to ensure only that tasks not execute concurrently and provide enough synchronization so that the memory effects of one task are guaranteed to be visible to the next task—which is precisely the guarantee offered by `newSingleThreadExecutor`.

Response-time-sensitive tasks. GUI applications are sensitive to response time: users are annoyed at long delays between a button click and the corresponding visual feedback. Submitting a long-running task to a single-threaded executor, or submitting several long-running tasks to a thread pool with a small number of threads, may impair the responsiveness of the service managed by that `Executor`.

Tasks that use `ThreadLocal`. `ThreadLocal` allows each thread to have its own private “version” of a variable. However, executors are free to reuse threads as they see fit. The standard `Executor` implementations may reap idle threads when demand is low and add new ones when demand is high, and also replace a worker thread with a fresh one if an unchecked exception is thrown from a task. `ThreadLocal` makes sense to use in pool threads only if the thread-local value has a lifetime that is bounded by that of a task; `ThreadLocal` should not be used in pool threads to communicate values between tasks.

Thread pools work best when tasks are *homogeneous* and *independent*. Mixing long-running and short-running tasks risks “clogging” the pool unless it is very large; submitting tasks that depend on other tasks risks deadlock unless the pool is unbounded. Fortunately, requests in typical network-based server applications—web servers, mail servers, file servers—usually meet these guidelines.

Some tasks have characteristics that require or preclude a specific execution policy. Tasks that depend on other tasks require that the thread pool be large enough that tasks are never queued or rejected; tasks that exploit thread confinement require sequential execution. Document these requirements so that future maintainers do not undermine safety or liveness by substituting an incompatible execution policy.

8.1.1. Thread Starvation Deadlock

If tasks that depend on other tasks execute in a thread pool, they can deadlock. In a single-threaded executor, a task that submits another task to the same executor and waits for its result will always deadlock. The second task sits on the work queue until the first task completes, but the first will not complete because it is waiting for the result of the second task. The same thing can happen in larger thread pools if all threads are executing tasks that are blocked waiting for other tasks still on the work queue. This is called *thread starvation deadlock*, and can occur whenever a pool task initiates an unbounded blocking wait for some resource or condition that can succeed only through the action of another pool task, such as waiting for the return value or side effect of another task, unless you can guarantee that the pool is large enough.

`ThreadDeadlock` in [Listing 8.1](#) illustrates thread starvation deadlock. `Render-PageTask` submits two additional tasks to the `Executor` to fetch the page header and footer, renders the page body, waits for the results of the header and footer tasks, and then combines the header, body, and footer into the finished page. With a single-threaded executor, `ThreadDeadlock` will always deadlock. Similarly, tasks coordinating amongst themselves with a barrier could also cause thread starvation deadlock if the pool is not big enough.

Whenever you submit to an `Executor` tasks that are not independent, be aware of the possibility of thread starvation deadlock, and document any pool sizing or configuration constraints in the code or configuration file where the `Executor` is configured.

In addition to any explicit bounds on the size of a thread pool, there may also be implicit limits because of constraints on other resources. If your application uses a JDBC connection pool with ten connections and each task needs a database connection, it is as if your thread pool only has ten threads because tasks in excess of ten will block waiting for a connection.

Listing 8.1. Task that Deadlocks in a Single-threaded `Executor` . *Don't do this.*

```
public class ThreadDeadlock {
    ExecutorService exec = Executors.newSingleThreadExecutor();

    public class RenderPageTask implements Callable<String> {
        public String call() throws Exception {
            Future<String> header, footer;
            header = exec.submit(new LoadFileTask("header.html"));
            footer = exec.submit(new LoadFileTask("footer.html"));
            String page = renderBody();
            // Will deadlock -- task waiting for result of subtask
            return header.get() + page + footer.get();
        }
    }
}
```



8.1.2. Long-running Tasks

Thread pools can have responsiveness problems if tasks can block for extended periods of time, even if deadlock is not a possibility. A thread pool can become clogged with long-running tasks, increasing the service time even for short tasks. If the pool size is too small relative to the expected steady-state number of longrunning tasks, eventually all the pool threads will be running long-running tasks and responsiveness will suffer.

One technique that can mitigate the ill effects of long-running tasks is for tasks to use timed resource waits instead of unbounded waits. Most blocking methods in the platform libraries come in both untimed and timed versions, such as `Thread.join`, `BlockingQueue.put`, `CountDownLatch.await`, and `Selector.select`. If the wait times out, you can mark the task as failed and abort it or requeue it for execution later. This guarantees that each task eventually makes progress towards either successful or failed completion, freeing up threads for tasks that

might complete more quickly. If a thread pool is frequently full of blocked tasks, this may also be a sign that the pool is too small.

8.2. Sizing Thread Pools

The ideal size for a thread pool depends on the types of tasks that will be submitted and the characteristics of the deployment system. Thread pool sizes should rarely be hard-coded; instead pool sizes should be provided by a configuration mechanism or computed dynamically by consulting `Runtime.availableProcessors`.

Sizing thread pools is not an exact science, but fortunately you need only avoid the extremes of “too big” and “too small”. If a thread pool is too big, then threads compete for scarce CPU and memory resources, resulting in higher memory usage and possible resource exhaustion. If it is too small, throughput suffers as processors go unused despite available work.

To size a thread pool properly, you need to understand your computing environment, your resource budget, and the nature of your tasks. How many processors does the deployment system have? How much memory? Do tasks perform mostly computation, I/O, or some combination? Do they require a scarce resource, such as a JDBC connection? If you have different categories of tasks with very different behaviors, consider using multiple thread pools so each can be tuned according to its workload.

For compute-intensive tasks, an N_{cpu} -processor system usually achieves optimum utilization with a thread pool of $N_{cpu} + 1$ threads. (Even compute-intensive threads occasionally take a page fault or pause for some other reason, so an “extra” runnable thread prevents CPU cycles from going unused when this happens.) For tasks that also include I/O or other blocking operations, you want a larger pool, since not all of the threads will be schedulable at all times. In order to size the pool properly, you must estimate the ratio of waiting time to compute time for your tasks; this estimate need not be precise and can be obtained through pro-filing or instrumentation. Alternatively, the size of the thread pool can be tuned by running the application using several different pool sizes under a benchmark load and observing the level of CPU utilization.

Given these definitions:

$$\begin{aligned} N_{cpu} &= \text{number of CPUs} \\ U_{cpu} &= \text{target CPU utilization, } 0 \leq U_{cpu} \leq 1 \\ \frac{W}{C} &= \text{ratio of wait time to compute time} \end{aligned}$$

The optimal pool size for keeping the processors at the desired utilization is:

$$N_{threads} = N_{cpu} * U_{cpu} * \left(1 + \frac{W}{C}\right)$$

You can determine the number of CPUs using `Runtime`:

```
int N_CPUS = Runtime.getRuntime().availableProcessors();
```

Of course, CPU cycles are not the only resource you might want to manage using thread pools. Other resources that can contribute to sizing constraints are memory, file handles, socket handles, and database connections. Calculating pool size constraints for these types of resources is easier: just add up how much of that resource each task requires and divide that into the total quantity available. The result will be an upper bound on the pool size.

When tasks require a pooled resource such as database connections, thread pool size and resource pool size affect each other. If each task requires a connection, the effective size of the thread pool is limited by the connection pool size. Similarly, when the only consumers of connections are pool tasks, the effective size of the connection pool is limited by the thread pool size.

8.3. Configuring `ThreadPoolExecutor`

`ThreadPoolExecutor` provides the base implementation for the executors returned by the `newCachedThreadPool`, `newFixedThreadPool`, and `newScheduledThreadPool` factories in `Executors`.

`ThreadPoolExecutor` is a flexible, robust pool implementation that allows a variety of customizations.

If the default execution policy does not meet your needs, you can instantiate a `ThreadPoolExecutor` through its constructor and customize it as you see fit; you can consult the source code for `Executors` to see the execution policies for the default configurations and use them as a starting point. `ThreadPoolExecutor` has several constructors, the most general of which is shown in [Listing 8.2](#).

8.3.1. Thread Creation and Teardown

The core pool size, maximum pool size, and keep-alive time govern thread creation and teardown. The core size is the target size; the implementation attempts to maintain the pool at this size even when there are no tasks to execute,^[2] and will not create more threads than this unless the work queue is full.^[3] The maximum pool size is the upper bound on how many pool threads can be active at once. A thread that has been idle for longer than the keep-alive time becomes a candidate for reaping and can be terminated if the current pool size exceeds the core size.

^[2] When a `ThreadPoolExecutor` is initially created, the core threads are not started immediately but instead as tasks are submitted, unless you call `prestartAllCoreThreads`.

^[3] Developers are sometimes tempted to set the core size to zero so that the worker threads will eventually be torn down and therefore won't prevent the JVM from exiting, but this can cause some strange-seeming behavior in thread pools that don't use a `SynchronousQueue` for their work queue (as `newCachedThreadPool` does). If the pool is already at the core size, `ThreadPoolExecutor` creates a new thread only if the work queue is full. So tasks submitted to a thread pool with a work queue that has any capacity and a core size of zero will not execute until the queue fills up, which is usually not what is desired. In Java 6, `allowCoreThreadTimeOut` allows you to request that all pool threads be able to time out; enable this feature with a core size of zero if you want a bounded thread pool with a bounded work queue but still have all the threads torn down when there is no work to do.

Listing 8.2. General Constructor for `ThreadPoolExecutor` .

```
public ThreadPoolExecutor(int corePoolSize,
                        int maximumPoolSize,
                        long keepAliveTime,
                        TimeUnit unit,
                        BlockingQueue<Runnable> workQueue,
                        ThreadFactory threadFactory,
                        RejectedExecutionHandler handler) { ... }
```

By tuning the core pool size and keep-alive times, you can encourage the pool to reclaim resources used by otherwise idle threads, making them available for more useful work. (Like everything else, this is a tradeoff: reaping idle threads incurs additional latency due to thread creation if threads must later be created when demand increases.)

The `newFixedThreadPool` factory sets both the core pool size and the maximum pool size to the requested pool size, creating the effect of infinite timeout; the `newCachedThreadPool` factory sets the maximum pool size to `Integer.MAX_VALUE` and the core pool size to zero with a timeout of one minute, creating the effect of an infinitely expandable thread pool that will contract again when demand decreases. Other combinations are possible using the explicit `ThreadPoolExecutor` constructor.

8.3.2. Managing Queued Tasks

Bounded thread pools limit the number of tasks that can be executed concurrently. (The single-threaded executors are a notable special case: they guarantee that no tasks will execute concurrently, offering the possibility of achieving thread safety through thread confinement.)

We saw in [Section 6.1.2](#) how unbounded thread creation could lead to instability, and addressed this problem by using a fixed-sized thread pool instead of creating a new thread for every request. However, this is only a partial solution; it is still possible for the application to run out of resources under heavy load, just harder. If the arrival rate for new requests exceeds the rate at which they can be handled, requests will still queue up. With a thread pool, they wait in a queue of `Runnable`s managed by the `Executor` instead of queueing up as threads contending for the CPU. Representing a waiting task with a `Runnable` and a list node is certainly

a lot cheaper than with a thread, but the risk of resource exhaustion still remains if clients can throw requests at the server faster than it can handle them.

Requests often arrive in bursts even when the average request rate is fairly stable. Queues can help smooth out transient bursts of tasks, but if tasks continue to arrive too quickly you will eventually have to throttle the arrival rate to avoid running out of memory.^[4] Even before you run out of memory, response time will get progressively worse as the task queue grows.

^[4] This is analogous to flow control in communications networks: you may be willing to buffer a certain amount of data, but eventually you need to find a way to get the other side to stop sending you data, or throw the excess data on the floor and hope the sender retransmits it when you're not so busy.

`ThreadPoolExecutor` allows you to supply a `BlockingQueue` to hold tasks awaiting execution. There are three basic approaches to task queueing: unbounded queue, bounded queue, and synchronous handoff. The choice of queue interacts with other configuration parameters such as pool size.

The default for `newFixedThreadPool` and `newSingleThreadExecutor` is to use an unbounded `LinkedBlockingQueue`. Tasks will queue up if all worker threads are busy, but the queue could grow without bound if the tasks keep arriving faster than they can be executed.

A more stable resource management strategy is to use a bounded queue, such as an `ArrayBlockingQueue` or a bounded `LinkedBlockingQueue` or `PriorityBlockingQueue`. Bounded queues help prevent resource exhaustion but introduce the question of what to do with new tasks when the queue is full. (There are a number of possible *saturation policies* for addressing this problem; see [Section 8.3.3](#).) With a bounded work queue, the queue size and pool size must be tuned together. A large queue coupled with a small pool can help reduce memory usage, CPU usage, and context switching, at the cost of potentially constraining throughput.

For very large or unbounded pools, you can also bypass queueing entirely and instead hand off tasks directly from producers to worker threads using a `SynchronousQueue`. A `SynchronousQueue` is not really a queue at all, but a mechanism for managing handoffs between threads. In order to put an element on a `SynchronousQueue`, another thread must already be waiting to accept the handoff. If no thread is waiting but the current pool size is less than the maximum, `ThreadPoolExecutor` creates a new thread; otherwise the task is rejected according to the saturation policy. Using a direct handoff is more efficient because the task can be handed right to the thread that will execute it, rather than first placing it on a queue and then having the worker thread fetch it from the queue.

`SynchronousQueue` is a practical choice only if the pool is unbounded or if rejecting excess tasks is acceptable. The `newCachedThreadPool` factory uses a `SynchronousQueue`.

Using a FIFO queue like `LinkedBlockingQueue` or `ArrayBlockingQueue` causes tasks to be started in the order in which they arrived. For more control over task execution order, you can use a `PriorityBlockingQueue`, which orders tasks according to priority. Priority can be defined by natural order (if tasks implement `Comparable`) or by a `Comparator`.

The `newCachedThreadPool` factory is a good default choice for an `Executor`, providing better queuing performance than a fixed thread pool.^[5] A fixed size thread pool is a good choice when you need to limit the number of concurrent tasks for resource-management purposes, as in a server application that accepts requests from network clients and would otherwise be vulnerable to overload.

[5] This performance difference comes from the use of `SynchronousQueue` instead of `LinkedBlockingQueue`. `SynchronousQueue` was replaced in Java 6 with a new nonblocking algorithm that improved throughput in `Executor` benchmarks by a factor of three over the Java 5.0 `SynchronousQueue` implementation ([Scherer et al., 2006](#)).

Bounding either the thread pool or the work queue is suitable only when tasks are independent. With tasks that depend on other tasks, bounded thread pools or queues can cause thread starvation deadlock; instead, use an unbounded pool configuration like `newCachedThreadPool`.^[6]

^[6] An alternative configuration for tasks that submit other tasks and wait for their results is to use a bounded thread pool, a `SynchronousQueue` as the work queue, and the caller-runs saturation policy.

8.3.3. Saturation Policies

When a bounded work queue fills up, the *saturation policy* comes into play. The saturation policy for a `ThreadPoolExecutor` can be modified by calling `setRejectedExecutionHandler`. (The saturation policy is also used when a task is submitted to an `Executor` that has been shut down.) Several implementations of `RejectedExecutionHandler` are provided, each implementing a different saturation policy:

`AbortPolicy`, `CallerRunsPolicy`, `DiscardPolicy`, and `DiscardOldestPolicy`.

The default policy, *abort*, causes `execute` to throw the unchecked `RejectedExecutionException`; the caller can catch this exception and implement its own overflow handling as it sees fit. The *discard* policy silently discards the newly submitted task if it cannot be queued for execution; the *discard-oldest* policy discards the task that would otherwise be executed next and tries to resubmit the new task. (If the work queue is a priority queue, this discards the highest-priority element, so the combination of a discard-oldest saturation policy and a priority queue is not a good one.)

The *caller-runs* policy implements a form of throttling that neither discards tasks nor throws an exception, but instead tries to slow down the flow of new tasks by pushing some of the work back to the caller. It executes the newly submitted task not in a pool thread, but in the thread that calls `execute`. If we modified our `WebServer` example to use a bounded queue and the caller-runs policy, after all the pool threads were occupied and the work queue filled up the next task would be executed in the

main thread during the call to `execute`. Since this would probably take some time, the main thread cannot submit any more tasks for at least a little while, giving the worker threads some time to catch up on the backlog. The main thread would also not be calling `accept` during this time, so incoming requests will queue up in the TCP layer instead of in the application. If the overload persisted, eventually the TCP layer would decide it has queued enough connection requests and begin discarding connection requests as well. As the server becomes overloaded, the overload is gradually pushed outward—from the pool threads to the work queue to the application to the TCP layer, and eventually to the client—enabling more graceful degradation under load.

Choosing a saturation policy or making other changes to the execution policy can be done when the `Executor` is created. [Listing 8.3](#) illustrates creating a fixedsize thread pool with the caller-runs saturation policy.

Listing 8.3. Creating a Fixed-sized Thread Pool with a Bounded Queue and the Caller-runs Saturation Policy.

```
ThreadPoolExecutor executor
    = new ThreadPoolExecutor(N_THREADS, N_THREADS,
        0L, TimeUnit.MILLISECONDS,
        new LinkedBlockingQueue<Runnable>(CAPACITY));
executor.setRejectedExecutionHandler(
    new ThreadPoolExecutor.CallerRunsPolicy());
```

There is no predefined saturation policy to make `execute` block when the work queue is full. However, the same effect can be accomplished by using a `Semaphore` to bound the task injection rate, as shown in `BoundedExecutor` in [Listing 8.4](#). In such an approach, use an unbounded queue (there's no reason to bound both the queue size and the injection rate) and set the bound on the semaphore to be equal to the pool size *plus* the number of queued tasks you want to allow, since the semaphore is bounding the number of tasks both currently executing and awaiting execution.

8.3.4. Thread Factories

Whenever a thread pool needs to create a thread, it does so through a *thread factory* (see [Listing 8.5](#)). The default thread factory creates a new,

nondaemon thread with no special configuration. Specifying a thread factory allows you to customize the configuration of pool threads.

`ThreadFactory` has a single method, `newThread`, that is called whenever a thread pool needs to create a new thread.

There are a number of reasons to use a custom thread factory. You might want to specify an `UncaughtExceptionHandler` for pool threads, or instantiate an instance of a custom `Thread` class, such as one that performs debug logging. You might want to modify the priority (generally not a very good idea; see [Section 10.3.1](#)) or set the daemon status (again, not all that good an idea; see [Section 7.4.2](#)) of pool threads. Or maybe you just want to give pool threads more meaningful names to simplify interpreting thread dumps and error logs.

Listing 8.4. Using a `Semaphore` to Throttle Task Submission.

```
@ThreadSafe
public class BoundedExecutor {
    private final Executor exec;
    private final Semaphore semaphore;

    public BoundedExecutor(Executor exec, int bound) {
        this.exec = exec;
        this.semaphore = new Semaphore(bound);
    }

    public void submitTask(final Runnable command)
        throws InterruptedException {
        semaphore.acquire();
        try {
            exec.execute(new Runnable() {
                public void run() {
                    try {
                        command.run();
                    } finally {
                        semaphore.release();
                    }
                }
            });
        } catch (RejectedExecutionException e) {
            semaphore.release();
        }
    }
}
```

Listing 8.5. `ThreadFactory` Interface.

```
public interface ThreadFactory {  
    Thread newThread(Runnable r);  
}
```

`MyThreadFactory` in [Listing 8.6](#) illustrates a custom thread factory. It instantiates a new `MyAppThread`, passing a pool-specific name to the constructor so that threads from each pool can be distinguished in thread dumps and error logs. `MyAppThread` can also be used elsewhere in the application so that all threads can take advantage of its debugging features.

Listing 8.6. Custom Thread Factory.

```
public class MyThreadFactory implements ThreadFactory {  
    private final String poolName;  
  
    public MyThreadFactory(String poolName) {  
        this.poolName = poolName;  
    }  
  
    public Thread newThread(Runnable runnable) {  
        return new MyAppThread(runnable, poolName);  
    }  
}
```

The interesting customization takes place in `MyAppThread`, shown in [Listing 8.7](#), which lets you provide a thread name, sets a custom `UncaughtExceptionHandler` that writes a message to a `Logger`, maintains statistics on how many threads have been created and destroyed, and optionally writes a debug message to the log when a thread is created or terminates.

If your application takes advantage of *security policies* to grant permissions to particular codebases, you may want to use the `privilegedThreadFactory` factory method in `Executors` to construct your thread factory. It creates pool threads that have the same permissions, `AccessControlContext`, and `contextClassLoader` as the thread creating the `privilegedThreadFactory`. Otherwise, threads created by the thread pool inherit permissions from whatever client happens to be calling `execute` or `submit` at the time a new thread is needed, which could cause confusing security-related exceptions.

8.3.5. Customizing ThreadPoolExecutor After Construction

Most of the options passed to the `ThreadPoolExecutor` constructors can also be modified after construction via setters (such as the core thread pool size, maximum thread pool size, keep-alive time, thread factory, and rejected execution handler). If the `Executor` is created through one of the factory methods in `Executors` (except `newSingleThreadExecutor`), you can cast the result to `ThreadPoolExecutor` to access the setters as in [Listing 8.8](#).

`Executors` includes a factory method, `unconfigurableExecutorService`, which takes an existing `ExecutorService` and wraps it with one exposing only the methods of `ExecutorService` so it cannot be further configured. Unlike the pooled implementations, `newSingleThreadExecutor` returns an `ExecutorService` wrapped in this manner, rather than a raw `ThreadPoolExecutor`. While a single-threaded executor is actually implemented as a thread pool with one thread, it also promises not to execute tasks concurrently. If some misguided code were to increase the pool size on a single-threaded executor, it would undermine the intended execution semantics.

Listing 8.7. Custom Thread Base Class.

```

public class MyAppThread extends Thread {
    public static final String DEFAULT_NAME = "MyAppThread";
    private static volatile boolean debugLifecycle = false;
    private static final AtomicInteger created = new AtomicInteger();
    private static final AtomicInteger alive = new AtomicInteger();
    private static final Logger log = Logger.getAnonymousLogger();

    public MyAppThread(Runnable r) { this(r, DEFAULT_NAME); }

    public MyAppThread(Runnable runnable, String name) {
        super(runnable, name + "-" + created.incrementAndGet());
        setUncaughtExceptionHandler(
            new Thread.UncaughtExceptionHandler() {
                public void uncaughtException(Thread t,
                                                Throwable e) {
                    log.log(Level.SEVERE,
                        "UNCAUGHT in thread " + t.getName(), e);
                }
            });
    }

    public void run() {
        // Copy debug flag to ensure consistent value throughout.
        boolean debug = debugLifecycle;
        if (debug) log.log(Level.FINE, "Created "+getName());
        try {
            alive.incrementAndGet();
            super.run();
        } finally {
            alive.decrementAndGet();
            if (debug) log.log(Level.FINE, "Exiting "+getName());
        }
    }

    public static int getThreadsCreated() { return created.get(); }
    public static int getThreadsAlive() { return alive.get(); }
    public static boolean getDebug() { return debugLifecycle; }
    public static void setDebug(boolean b) { debugLifecycle = b; }
}

```

Listing 8.8. Modifying an `Executor` created with the standard factories.

```

ExecutorService exec = Executors.newCachedThreadPool();
if (exec instanceof ThreadPoolExecutor)
    ((ThreadPoolExecutor) exec).setCorePoolSize(10);
else
    throw new AssertionError("Oops, bad assumption");

```

You can use this technique with your own executors to prevent the execution policy from being modified. If you will be exposing an `ExecutorService` to code you don't trust not to modify it, you can wrap it with an `unconfigurableExecutorService`.

8.4. Extending `ThreadPoolExecutor`

`ThreadPoolExecutor` was designed for extension, providing several “hooks” for subclasses to override—`beforeExecute`, `afterExecute`, and `terminated`—that can be used to extend the behavior of `ThreadPoolExecutor`.

The `beforeExecute` and `afterExecute` hooks are called in the thread that executes the task, and can be used for adding logging, timing, monitoring, or statistics gathering. The `afterExecute` hook is called whether the task completes by returning normally from `run` or by throwing an `Exception`. (If the task completes with an `Error`, `afterExecute` is not called.) If `beforeExecute` throws a `RuntimeException`, the task is not executed and `afterExecute` is not called.

The `terminated` hook is called when the thread pool completes the shutdown process, after all tasks have finished and all worker threads have shut down. It can be used to release resources allocated by the `Executor` during its lifecycle, perform notification or logging, or finalize statistics gathering.

8.4.1. Example: adding statistics to a thread pool

`TimingThreadPool` in [Listing 8.9](#) shows a custom thread pool that uses `beforeExecute`, `afterExecute`, and `terminated` to add logging and statistics gathering. To measure a task's runtime, `beforeExecute` must record the start time and store it somewhere `afterExecute` can find it. Because execution hooks are called in the thread that executes the task, a value placed in a `ThreadLocal` by `beforeExecute` can be retrieved by `afterExecute`. `TimingThreadPool` uses a pair of `AtomicLong`s to keep track of the total number of tasks processed and the total processing time, and uses the `terminated` hook to print a log message showing the average task time.

Listing 8.9. Thread Pool Extended with Logging and Timing.

```

public class TimingThreadPool extends ThreadPoolExecutor {
    private final ThreadLocal<Long> startTime
        = new ThreadLocal<Long>();
    private final Logger log = Logger.getLogger("TimingThreadPool");
    private final AtomicLong numTasks = new AtomicLong();
    private final AtomicLong totalTime = new AtomicLong();

    protected void beforeExecute(Thread t, Runnable r) {
        super.beforeExecute(t, r);
        log.fine(String.format("Thread %s: start %s", t, r));
        startTime.set(System.nanoTime());
    }

    protected void afterExecute(Runnable r, Throwable t) {
        try {
            long endTime = System.nanoTime();
            long taskTime = endTime - startTime.get();
            numTasks.incrementAndGet();
            totalTime.addAndGet(taskTime);
            log.fine(String.format("Thread %s: end %s, time=%dns",
                                   t, r, taskTime));
        } finally {
            super.afterExecute(r, t);
        }
    }

    protected void terminated() {
        try {
            log.info(String.format("Terminated: avg time=%dns",
                                   totalTime.get() / numTasks.get()));
        } finally {
            super.terminated();
        }
    }
}

```

8.5. Parallelizing Recursive Algorithms

The page rendering examples in [Section 6.3](#) went through a series of refinements in search of exploitable parallelism. The first attempt was entirely sequential; the second used two threads but still performed all the image downloads sequentially; the final version treated each image download as a separate task to achieve greater parallelism. Loops whose bodies contain nontrivial computation or perform potentially blocking I/O are frequently good candidates for parallelization, as long as the iterations are independent.

If we have a loop whose iterations are independent and we don't need to wait for all of them to complete before proceeding, we can use an

`Executor` to transform a sequential loop into a parallel one, as shown in `processSequentially` and `processInParallel` in [Listing 8.10](#).

Listing 8.10. Transforming Sequential Execution into Parallel Execution.

```
void processSequentially(List<Element> elements) {
    for (Element e : elements)
        process(e);
}

void processInParallel(Executor exec, List<Element> elements) {
    for (final Element e : elements)
        exec.execute(new Runnable() {
            public void run() { process(e); }
        });
}
```

A call to `processInParallel` returns more quickly than a call to `processSequentially` because it returns as soon as all the tasks are queued to the `Executor`, rather than waiting for them all to complete. If you want to submit a set of tasks and wait for them all to complete, you can use `ExecutorService.invokeAll`; to retrieve the results as they become available, you can use a `CompletionService`, as in `Renderer` on page [130](#).

Sequential loop iterations are suitable for parallelization when each iteration is independent of the others and the work done in each iteration of the loop body is significant enough to offset the cost of managing a new task.

Loop parallelization can also be applied to some recursive designs; there are often sequential loops within the recursive algorithm that can be parallelized in the same manner as [Listing 8.10](#). The easier case is when each iteration does not require the results of the recursive iterations it invokes. For example, `sequentialRecursive` in [Listing 8.11](#) does a depth-first traversal of a tree, performing a calculation on each node and placing the result in a collection. The transformed version, `parallelRecursive`, also does a depth-first traversal, but instead of computing the result as each node is visited, it submits a task to compute the node result.

Listing 8.11. Transforming Sequential Tail-recursion into Parallelized Recursion.

```

public<T> void sequentialRecursive(List<Node<T>> nodes,
                                   Collection<T> results) {
    for (Node<T> n : nodes) {
        results.add(n.compute());
        sequentialRecursive(n.getChildren(), results);
    }
}

public<T> void parallelRecursive(final Executor exec,
                                List<Node<T>> nodes,
                                final Collection<T> results) {
    for (final Node<T> n : nodes) {
        exec.execute(new Runnable() {
            public void run() {
                results.add(n.compute());
            }
        });
        parallelRecursive(exec, n.getChildren(), results);
    }
}

```

When `parallelRecursive` returns, each node in the tree has been visited (the traversal is still sequential: only the calls to `compute` are executed in parallel) and the computation for each node has been queued to the `Executor`. Callers of `parallelRecursive` can wait for all the results by creating an `Executor` specific to the traversal and using `shutdown` and `awaitTermination`, as shown in [Listing 8.12](#).

Listing 8.12. Waiting for Results to be Calculated in Parallel.

```

public<T> Collection<T> getParallelResults(List<Node<T>> nodes)
    throws InterruptedException {
    ExecutorService exec = Executors.newCachedThreadPool();
    Queue<T> resultQueue = new ConcurrentLinkedQueue<T>();
    parallelRecursive(exec, nodes, resultQueue);
    exec.shutdown();
    exec.awaitTermination(Long.MAX_VALUE, TimeUnit.SECONDS);
    return resultQueue;
}

```

8.5.1. Example: A Puzzle Framework

An appealing application of this technique is solving puzzles that involve finding a sequence of transformations from some initial state to reach a

goal state, such as the familiar “sliding block puzzles”,^[7] “Hi-Q”, “Instant Insanity”, and other solitaire puzzles.

[7] See <http://www.puzzleworld.org/SlidingBlockPuzzles> .

We define a “puzzle” as a combination of an initial position, a goal position, and a set of rules that determine valid moves. The rule set has two parts: computing the list of legal moves from a given position and computing the result of applying a move to a position. `Puzzle` in [Listing 8.13](#) shows our puzzle abstraction; the type parameters `P` and `M` represent the classes for a position and a move. From this interface, we can write a simple sequential solver that searches the puzzle space until a solution is found or the puzzle space is exhausted.

Listing 8.13. Abstraction for Puzzles Like the “Sliding Blocks Puzzle”.

```
public interface Puzzle<P, M> {  
    P initialPosition();  
    boolean isGoal(P position);  
    Set<M> legalMoves(P position);  
    P move(P position, M move);  
}
```

`Node` in [Listing 8.14](#) represents a position that has been reached through some series of moves, holding a reference to the move that created the position and the previous `Node` . Following the links back from a `Node` lets us reconstruct the sequence of moves that led to the current position.

`SequentialPuzzleSolver` in [Listing 8.15](#) shows a sequential solver for the puzzle framework that performs a depth-first search of the puzzle space. It terminates when it finds a solution (which is not necessarily the shortest solution).

Rewriting the solver to exploit concurrency would allow us to compute next moves and evaluate the goal condition in parallel, since the process of evaluating one move is mostly independent of evaluating other moves. (We say “mostly” because tasks share some mutable state, such as the set of seen positions.) If multiple processors are available, this could reduce the time it takes to find a solution.

`ConcurrentPuzzleSolver` in [Listing 8.16](#) uses an inner `SolverTask` class that extends `Node` and implements `Runnable`. Most of the work is done in `run`: evaluating the set of possible next positions, pruning positions already searched, evaluating whether success has yet been achieved (by this task or by some other task), and submitting unsearched positions to an `Executor`.

To avoid infinite loops, the sequential version maintained a `Set` of previously searched positions; `ConcurrentPuzzleSolver` uses a `ConcurrentHashMap` for this purpose. This provides thread safety and avoids the race condition inherent in conditionally updating a shared collection by using `putIfAbsent` to atomically add a position only if it was not previously known. `ConcurrentPuzzleSolver` uses the internal work queue of the thread pool instead of the call stack to hold the state of the search.

Listing 8.14. Link Node for the Puzzle Solver Framework.

```
@Immutable
static class Node<P, M> {
    final P pos;
    final M move;
    final Node<P, M> prev;

    Node(P pos, M move, Node<P, M> prev) {...}

    List<M> asMoveList() {
        List<M> solution = new LinkedList<M>();
        for (Node<P, M> n = this; n.move != null; n = n.prev)
            solution.add(0, n.move);
        return solution;
    }
}
```

The concurrent approach also trades one form of limitation for another that might be more suitable to the problem domain. The sequential version performs a depth-first search, so the search is bounded by the available stack size. The concurrent version performs a breadth-first search and is therefore free of the stack size restriction (but can still run out of memory if the set of positions to be searched or already searched exceeds the available memory).

In order to stop searching when we find a solution, we need a way to determine whether any thread has found a solution yet. If we want to accept the first solution found, we also need to update the solution only if no other task has already found one. These requirements describe a sort of *latch* (see [Section 5.5.1](#)) and in particular, a *result-bearing latch*. We could easily build a blocking resultbearing latch using the techniques in [Chapter 14](#), but it is often easier and less error-prone to use existing library classes rather than low-level language mechanisms. `ValueLatch` in [Listing 8.17](#) uses a `CountDownLatch` to provide the needed latching behavior, and uses locking to ensure that the solution is set only once.

Each task first consults the solution latch and stops if a solution has already been found. The main thread needs to wait until a solution is found; `getValue` in `ValueLatch` blocks until some thread has set the value. `ValueLatch` provides a way to hold a value such that only the first call actually sets the value, callers can test whether it has been set, and callers can block waiting for it to be set. On the first call to `setValue`, the solution is updated and the `CountDownLatch` is decremented, releasing the main solver thread from `getValue`.

The first thread to find a solution also shuts down the `Executor`, to prevent new tasks from being accepted. To avoid having to deal with `RejectedExecutionException`, the rejected execution handler should be set to discard submitted tasks. Then, all unfinished tasks eventually run to completion and any subsequent attempts to execute new tasks fail silently, allowing the executor to terminate. (If the tasks took longer to run, we might want to interrupt them instead of letting them finish.)

Listing 8.15. Sequential Puzzle Solver.

```
public class SequentialPuzzleSolver<P, M> {
    private final Puzzle<P, M> puzzle;
    private final Set<P> seen = new HashSet<P>();

    public SequentialPuzzleSolver(Puzzle<P, M> puzzle) {
        this.puzzle = puzzle;
    }

    public List<M> solve() {
        P pos = puzzle.initialPosition();
        return search(new Node<P, M>(pos, null, null));
    }

    private List<M> search(Node<P, M> node) {
        if (!seen.contains(node.pos)) {
            seen.add(node.pos);
            if (puzzle.isGoal(node.pos))
                return node.asMoveList();
            for (M move : puzzle.legalMoves(node.pos)) {
                P pos = puzzle.move(node.pos, move);
                Node<P, M> child = new Node<P, M>(pos, move, node);
                List<M> result = search(child);
                if (result != null)
                    return result;
            }
        }
        return null;
    }

    static class Node<P, M> { /* Listing 8.14 */ }
}
```

Listing 8.16. Concurrent Version of Puzzle Solver.

```

public class ConcurrentPuzzleSolver<P, M> {
    private final Puzzle<P, M> puzzle;
    private final ExecutorService exec;
    private final ConcurrentMap<P, Boolean> seen;
    final ValueLatch<Node<P, M>> solution
        = new ValueLatch<Node<P, M>>();
    ...
    public List<M> solve() throws InterruptedException {
        try {
            P p = puzzle.initialPosition();
            exec.execute(newTask(p, null, null));
            // block until solution found
            Node<P, M> solnNode = solution.getValue();
            return (solnNode == null) ? null : solnNode.asMoveList();
        } finally {
            exec.shutdown();
        }
    }

    protected Runnable newTask(P p, M m, Node<P,M> n) {
        return new SolverTask(p, m, n);
    }

    class SolverTask extends Node<P, M> implements Runnable {
        ...
        public void run() {
            if (solution.isSet()
                || seen.putIfAbsent(pos, true) != null)
                return; // already solved or seen this position
            if (puzzle.isGoal(pos))
                solution.setValue(this);
            else
                for (M m : puzzle.legalMoves(pos))
                    exec.execute(
                        newTask(puzzle.move(pos, m), m, this));
        }
    }
}

```

**Listing 8.17. Result-bearing Latch Used by
ConcurrentPuzzleSolver.**

```

@ThreadSafe
public class ValueLatch<T> {
    @GuardedBy("this") private T value = null;
    private final CountDownLatch done = new CountDownLatch(1);

    public boolean isSet() {
        return (done.getCount() == 0);
    }

    public synchronized void setValue(T newValue) {
        if (!isSet()) {
            value = newValue;
            done.countDown();
        }
    }

    public T getValue() throws InterruptedException {
        done.await();
        synchronized (this) {
            return value;
        }
    }
}

```

`ConcurrentPuzzleSolver` does not deal well with the case where there is no solution: if all possible moves and positions have been evaluated and no solution has been found, `solve` waits forever in the call to `getSolution`. The sequential version terminated when it had exhausted the search space, but getting concurrent programs to terminate can sometimes be more difficult. One possible solution is to keep a count of active solver tasks and set the solution to null when the count drops to zero, as in [Listing 8.18](#).

Finding the solution may also take longer than we are willing to wait; there are several additional termination conditions we could impose on the solver. One is a time limit; this is easily done by implementing a timed `getValue` in `ValueLatch` (which would use the timed version of `await`), and shutting down the `Executor` and declaring failure if `getValue` times out. Another is some sort of puzzle-specific metric such as searching only up to a certain number of positions. Or we can provide a cancellation mechanism and let the client make its own decision about when to stop searching.

Listing 8.18. Solver that Recognizes when No Solution Exists.


```

public class PuzzleSolver<P,M> extends ConcurrentPuzzleSolver<P,M> {
    ...
    private final AtomicInteger taskCount = new AtomicInteger(0);

    protected Runnable newTask(P p, M m, Node<P,M> n) {
        return new CountingSolverTask(p, m, n);
    }

    class CountingSolverTask extends SolverTask {
        CountingSolverTask(P pos, M move, Node<P, M> prev) {
            super(pos, move, prev);
            taskCount.incrementAndGet();
        }
        public void run() {
            try {
                super.run();
            } finally {
                if (taskCount.decrementAndGet() == 0)
                    solution.setValue(null);
            }
        }
    }
}

```

Summary

The `Executor` framework is a powerful and flexible framework for concurrently executing tasks. It offers a number of tuning options, such as policies for creating and tearing down threads, handling queued tasks, and what to do with excess tasks, and provides several hooks for extending its behavior. As in most powerful frameworks, however, there are combinations of settings that do not work well together; some types of tasks require specific execution policies, and some combinations of tuning parameters may produce strange results.